

1. What is Azure Kubernetes Service (AKS), and how is it different from self-managed Kubernetes?

Azure Kubernetes Service (AKS) is a managed Kubernetes offering from Azure that simplifies cluster provisioning, upgrades, scaling, and security.

Unlike self-managed Kubernetes on VMs, AKS abstracts the control plane management, handles auto-healing, integrates with Azure AD, and offers built-in monitoring/logging with Azure Monitor and Log Analytics.

Interview-way of Answering:

In one of my recent projects, we migrated our containerized workloads to Azure Kubernetes Service (AKS) to take advantage of its managed infrastructure.

Compared to setting up Kubernetes on virtual machines, AKS made it much easier to manage the control plane, as Azure handles that completely including upgrades and high availability.

We didn't have to manually manage etcd or the API server like we would in a self-managed setup.

Plus, AKS integrates really well with other Azure services like Azure Monitor, Key Vault, and Azure Active Directory, which helped us enhance security and observability with minimal configuration.

2. How do you create a basic AKS cluster using Azure CLI?

```
az aks create \
  --resource-group myResourceGroup \
  --name myAKSCluster \
  --node-count 3 \
  --enable-addons monitoring \
  --generate-ssh-keys
```

This command creates a 3-node AKS cluster with monitoring enabled and generates SSH keys for node access.

Interview-way of Answering:

During the initial setup of our development environment, I used Azure CLI to provision the AKS cluster.

The process was straightforward.

I ran a script that used the `az aks create` command with parameters for node count, resource group, and enabled monitoring.

It also generated SSH keys automatically. Once the cluster was up, I retrieved the kubeconfig using `az aks get-credentials`, which allowed us to interact with the cluster using `kubectl`.

This setup helped our developers start deploying microservices within an hour.

3. What is a node pool in AKS, and why would you use multiple node pools?

A node pool in AKS is a group of VMs with the same configuration.

Multiple node pools allow workload separation (e.g., Linux and Windows containers), performance optimization (e.g., GPU-enabled pools for ML workloads), and cost control (e.g., using spot instances for non-critical workloads).

Interview-way of Answering:

In one of our production setups, we used multiple node pools in AKS to segregate workloads.

For example, we had a general-purpose node pool for our standard microservices and a separate GPU-enabled node pool for machine learning inference jobs.

This setup not only optimized resource allocation but also helped us apply taints and tolerations to ensure workloads were scheduled appropriately.

We also used spot instances in a separate pool for non-critical background jobs to reduce costs.

4. How does AKS handle Kubernetes upgrades?

AKS supports in-place, rolling upgrades of the Kubernetes version.

You can upgrade the control plane first, then the node pools gradually.

Azure ensures minimal disruption using a cordon and drain strategy for each node.

You can automate this using `az aks upgrade`.

Interview-way of Answering:

When managing our production AKS cluster, we followed a controlled upgrade process.

Azure allows us to upgrade the control plane first and then upgrade node pools one by one.

During the upgrade, Azure drains each node gracefully using the `kubectl drain` equivalent, which ensured zero downtime.

We usually scheduled upgrades during off-peak hours and verified all workloads with health probes and pre-deployment readiness checks.

This way, we avoided breaking the application while still keeping the cluster up to date.

5. How do you authenticate to an AKS cluster using Azure AD?

AKS supports Azure AD integration for Kubernetes RBAC. Users authenticate using `az login`, and then run:

```
az aks get-credentials --name <cluster-name> --resource-group <resource-group>
kubectl get pods
```

RBAC roles (e.g., `ClusterRoleBindings`) map Azure AD groups to Kubernetes permissions.

Interview-way of Answering:

Security was a priority in our AKS setup, so we integrated Azure AD for authentication.

This meant that our team members didn't need separate Kubernetes credentials they could authenticate via `az login`. Once authenticated, we used `az aks get-credentials` to access the cluster.

We then set up Kubernetes `RoleBindings` and `ClusterRoleBindings` to map Azure AD groups to specific roles.

For instance, developers had read-only access, while DevOps engineers had full admin rights.

This approach aligned perfectly with our organization's RBAC policies.

6. What are the best practices for managing secrets in AKS?

- Use **Azure Key Vault** with **CSI Secrets Store driver** for securely injecting secrets.
- Avoid hardcoding secrets in ConfigMaps or Deployment manifests.
- Use pod identity (AAD Pod Identity or workload identity) to access Key Vault securely.
- Rotate secrets periodically and audit access.

Interview-way of Answering:

In one of our secure environments, we had a strict policy of not storing secrets in Kubernetes secrets or config maps directly.

Instead, we integrated Azure Key Vault using the CSI Secrets Store driver. Our application pods mounted secrets from Key Vault directly as volumes, and we used Azure Workload Identity to allow those pods to access the vault without embedding credentials.

This method ensured that secrets like database passwords and API tokens were always pulled securely at runtime and rotated automatically without downtime.

7. How do you enable monitoring and logging in AKS?

Use **Azure Monitor for Containers**, which integrates with Log Analytics. Enable during cluster creation or afterward:

```
az aks enable-addons --addons monitoring --name myAKSCluster --resource-group myRG
```

Logs, metrics, and performance data become available in Log Analytics and Grafana via Azure Monitor.

Interview-way of Answering:

For observability, we enabled Azure Monitor for Containers right when we created the AKS cluster.

This provided built-in integration with Log Analytics, where all our container logs, metrics, and performance data were aggregated.

We could easily search logs, set up alerts, and visualize metrics using Azure dashboards and Grafana.

For troubleshooting production issues, this setup was invaluable for example, if a pod crashed, we could quickly trace the exact error logs in Log Analytics without SSH-ing into nodes.

8. Explain a real-time use case where you used Helm in AKS.

In one project, we used Helm to deploy Prometheus and Grafana into AKS:

```
helm repo add prometheus-community https://prometheus-community.github.io/helm-charts  
helm install monitoring prometheus-community/kube-prometheus-stack
```

This allowed us to template and manage complex Kubernetes manifests with values overrides for different environments.

Interview-way of Answering:

We used Helm extensively in our CI/CD pipelines for AKS.

One example was deploying Prometheus and Grafana using the kube-prometheus-stack Helm chart.

Instead of writing dozens of YAML files, we maintained values.yaml files for different environments dev, staging, and prod. During deployments, Helm allowed us to parameterize things like resource limits, alert rules, and dashboard configs.

This approach helped us standardize our deployments, reduce errors, and make rollbacks seamless using helm rollback.

9. How would you implement a blue-green deployment in AKS?

Use multiple versions of the application behind different services or selectors (e.g., app=blue, app=green). Route traffic using:

- Ingress Controller annotations.
- Azure Application Gateway Ingress Controller (AGIC).
- Manual switch or automation pipeline using labels/selectors update.

Interview-way of Answering:

We implemented blue-green deployments using Kubernetes Deployments and Ingress.

We had two separate deployments for the same app versioned as "blue" and "green", each labeled and routed through different Ingress rules.

Initially, traffic was routed to the “blue” deployment.

When deploying a new version, we rolled it out to the “green” environment and tested everything thoroughly. Once validated, we updated the Ingress to point to the “green” deployment.

This gave us near-zero downtime and the ability to quickly revert if something broke.

10. What is the difference between ClusterIP, NodePort, LoadBalancer, and Ingress in AKS?

Type	Description
ClusterIP	Default; internal access within cluster only.
NodePort	Opens a static port on each node’s IP; limited scalability.
LoadBalancer	Provisions Azure Load Balancer with a public IP; ideal for external apps.
Ingress	Manages routing using host/path rules; works with a single IP and TLS termination.

Interview-way of Answering:

In our AKS applications, we used all four types based on the use case.

For internal services like Redis or internal APIs, we used ClusterIP.

For debugging or ad-hoc access during development, we temporarily used NodePort.

For externally exposed apps, we used LoadBalancer, which Azure provisioned automatically via an external IP.

Over time, to consolidate routing and support TLS termination, we adopted Ingress Controllers like NGINX and Azure Application Gateway.

Ingress helped us manage multiple services under a single DNS with cleaner configurations and SSL support.

11. How do you implement network security in AKS?

- Use **Azure Network Policies** to restrict pod-to-pod communication.
- Apply **NSGs (Network Security Groups)** to AKS subnets.
- Use **Private AKS cluster** to restrict API server exposure.
- Integrate with **Azure Firewall** or **WAF** for advanced ingress filtering.

Interview-way of Answering:

In a previous project where we hosted a retail e-commerce backend on AKS, managing scalability was crucial during high traffic events like flash sales.

We enabled **Cluster Autoscaler** in AKS to dynamically add or remove nodes based on workload demands.

It scaled the number of nodes depending on pending pods or resource usage.

At the pod level, we implemented **Horizontal Pod Autoscaler (HPA)** for our microservices using metrics like CPU and custom business metrics (like queue length from Azure Monitor).

For example, our order-processing service scaled from 3 to 10 pods during peak loads.

This dual-layer autoscaling ensured optimal performance and cost-efficiency without manual intervention.

12. What is Azure CNI, and how is it different from Kubenet?

- **Azure CNI** assigns **IP addresses from the VNet** directly to pods, enabling full VNet integration and secure service-to-service communication.
- **Kubenet** uses NAT and is more limited in VNet integration. Azure CNI is better for enterprise-level networking with tight security requirements.

Interview-way of Answering:

In our CI/CD pipeline, we used GitHub Actions for building and deploying Dockerized microservices to AKS.

When a developer pushed code to the main branch, GitHub Actions triggered a workflow that built the image, pushed it to Azure Container Registry (ACR), and deployed it to AKS using kubectl and Helm.

We used secrets stored in GitHub Actions for authenticating with Azure via service principals. For multi-environment deployment (dev → stage → prod), we used branch-based workflows, Helm values overrides, and approvals for production.

This automated CI/CD setup significantly reduced manual errors and gave us faster feedback loops.

13. How do you implement pod autoscaling in AKS?

Enable **Horizontal Pod Autoscaler (HPA)** based on CPU/memory or custom metrics:

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
spec:
  minReplicas: 2
  maxReplicas: 10
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          averageUtilization: 60
```

You can also enable **Cluster Autoscaler** for node scaling:

```
az aks update --enable-cluster-autoscaler --min-count 1 --max-count 5
```

Interview-way of Answering:

Security was non-negotiable in our setup, so we enforced **network policies** using Calico in AKS.

For instance, we restricted access to our PostgreSQL service to only the backend API pod.

We defined network policies in YAML that allowed traffic only from specific pods with matching labels.

Additionally, we enabled **private cluster mode** in AKS to restrict API server access to only trusted networks.

We also deployed **Azure Firewall** and **NSGs** to control ingress/egress at the subnet level, ensuring tight perimeter security.

14. Describe a real-time troubleshooting scenario in AKS.

Pods were stuck in ImagePullBackOff. After checking:

```
kubectl describe pod <name>
```

We found the issue was due to an expired image pull secret. Updated it using:

```
kubectl create secret docker-registry myregistrykey ...
```

Then re-deployed the affected pods to resolve the issue.

Interview-way of Answering:

We used **Azure Application Gateway Ingress Controller (AGIC)** for production traffic in AKS.

It gave us native Azure integration and WAF (Web Application Firewall) support.

For internal workloads, we used NGINX Ingress Controller for flexibility.

For TLS, we used **Let's Encrypt** with **cert-manager** for automatic certificate provisioning and renewal.

Each application had its own certificate managed through Kubernetes Ingress annotations.

This eliminated manual certificate renewal headaches and ensured secure communication.

15. How do you use Azure DevOps or GitHub Actions to deploy to AKS?

Use azure/login, azure/aks-set-context, and kubectl actions in your pipeline:

- uses: azure/login@v1
with:
 creds: \${{ secrets.AZURE_CREDENTIALS }}
- uses: azure/aks-set-context@v1
with:
 resource-group: myRG
 cluster-name: myAKSCluster
- run: kubectl apply -f deployment.yaml

Interview-way of Answering:

Disaster recovery was part of our compliance requirements, especially for financial services workloads.

We designed a DR strategy with **daily etcd snapshots** (for internal test clusters) and **regular backups of critical workloads** using **Velero**.

We backed up Kubernetes resources like deployments, PVCs, config maps, and secrets.

We also used **Azure Backup** for underlying disk snapshots and replicated container images across geo-paired ACRs.

For faster recovery, we maintained infrastructure-as-code templates in Terraform to quickly spin up identical clusters in a secondary region in case of region-level failure.

16. How would you configure a private AKS cluster with secure DevOps access?

- Enable **private cluster** during creation.
- Use **Jumpbox (bastion host)** inside VNet to access the cluster.

- Set up **Private DNS Zone** for API server resolution.
- Use **Azure DevOps Private Agent** in the same VNet/subnet.

Interview-way of Answering:

In a large organization setup, we had multiple teams developers, QA, SREs accessing the same AKS cluster.

We used **Azure AD integration** to control authentication, and then Kubernetes **Role-Based Access Control (RBAC)** to assign fine-grained permissions.

For instance, developers were restricted to their own namespaces with read/write permissions on deployments and pods, but no access to Secrets or other namespaces.

We mapped Azure AD groups to Kubernetes roles using RoleBindings and ClusterRoleBindings.

This approach made it easy to audit and rotate access, especially during onboarding/offboarding.

17. What's the difference between user-assigned and system-assigned managed identities in AKS?

- **System-assigned:** Tied to the lifecycle of the AKS resource.
- **User-assigned:** Can be reused across multiple services, ideal for consistent access controls, e.g., accessing Key Vault from multiple workloads.

Interview-way of Answering:

For real-time monitoring, we used **Azure Monitor for Containers** which gave us deep visibility into node and pod health.

We could track resource usage, failed pods, and node disk pressure using built-in dashboards.

It also integrated with **Log Analytics**, where we set up custom alerts based on KQL queries.

Additionally, for open-source monitoring, we deployed **Prometheus and Grafana** using Helm.

We created custom Grafana dashboards for pod CPU/memory, node status, and even app-specific metrics using Prometheus exporters.

This hybrid observability stack helped us catch issues before they escalated.

18. How do you manage multi-tenant Kubernetes workloads in AKS?

- Use **namespaces** for logical separation.
- Apply **resource quotas** and **limit ranges**.
- Enforce **network policies** for tenant isolation.
- Integrate with **Azure AD** for namespace-specific RBAC.

Interview-way of Answering:

We deployed several stateful services — like MongoDB, PostgreSQL, and RabbitMQ — using Kubernetes **StatefulSets**.

To ensure persistent storage, we used **Azure Disks** via PersistentVolumeClaims.

Each pod in the StatefulSet got a stable hostname and a unique persistent volume, which helped maintain data consistency.

For high availability, we often used managed alternatives like **Azure Database for PostgreSQL**, but when self-hosting was necessary (like for RabbitMQ with custom config), we ensured anti-affinity rules, PVCs, and node selectors were properly configured to avoid single points of failure.

19. How can you secure ingress traffic in AKS using TLS?

- Use **Ingress Controller** (NGINX or AGIC).
- Store TLS certificates in **Azure Key Vault** or as **Kubernetes secrets**.
- Reference them in Ingress objects:

tls:

- hosts:

- myapp.example.com

secretName: my-tls-secret

- Automate using **cert-manager** with Let's Encrypt for auto-renewal.

Interview-way of Answering:

For advanced rollout strategies, we used **Istio** as our service mesh in AKS.

For canary deployments, we used Istio VirtualServices to route 90% of traffic to the stable version and 10% to the new version.

We monitored metrics and logs for the canary pods — if everything was healthy, we gradually increased traffic.

This approach gave us much more control than standard Ingress-based routing.

We even used it for A/B testing features in staging.

The added benefit of mTLS and policy enforcement via Istio helped us standardize internal service communication.

20. How do you implement disaster recovery and backup for AKS workloads?

- Use **Velero** to back up volumes, Kubernetes resources, and restore in another region.
- Backup container images in **Azure Container Registry**.
- Use **Azure Backup** for persistent volumes (Azure Disk/File).
- Store manifests and Helm charts in Git (GitOps).

Interview-way of Answering:

In a cost-sensitive project, we followed several practices to optimize AKS spend.

First, we used **spot node pools** for non-critical batch jobs, reducing compute costs significantly.

We also set **resource requests and limits** carefully to avoid over-provisioning.

Another tactic was **using auto-scaling down to zero** in dev environments overnight using `az aks stop`.

We also monitored node and pod utilization through Azure Advisor and reduced unused services.

Over time, this brought our AKS costs down by ~30% while maintaining performance.